

3장 공간 데이터 확보, 필요한 조각만 받고 어디까지 믿을지 판정하기

- 이 장에 나오는 검색 결과 개수, 파일 크기, 읽기 시간, 포착률, 확률 값은 모두 `lecture_practice/chapter3/` 의 코드를 실제로 실행해 얻은 값
- 예시를 위해 지어낸 숫자는 없음

1. 이 장의 핵심 질문

☐ 이 장은 세 가지 질문에 답함

○ **질문 1** — 위성영상은 페타바이트 단위로 쌓여 있음. 그중 내 분석에 필요한 조각만 어떻게 찾아서, 어떻게 그 조각만 받는가?

○ **질문 2** — "지도에 없다"와 "현실에 없다"는 같은 말인가? 다르다면 그 차이를 어떻게 숫자로 재는가?

○ **질문 3** — 빠짐이 많은 지도에서 "0개"라는 관측을 근거로 결정을 내려도 되는가? 되는지 안 되는지는 무엇이 정하는가?

○ 질문 1은 자료를 손에 넣는 절차이고, 질문 2·3은 손에 넣은 자료를 어디까지 믿을지 판정하는 절차임

○ 이 장의 결론을 한 문장으로 먼저 적으면 — "자료에 없다"와 "현실에 없다"를 구분하지 못하면, 공공은 이미 시설이 있는 곳에 예산을 보내고 기업은 짝 찬 상권을 빈 시장으로 오판함

2. 지도에 카페가 두 곳뿐인 동네

○ 어느 행정동의 카페가 지도 자료에 두 곳으로 집계됨

☐ 이 관측을 읽는 방법은 둘

1. 실제로 두 곳뿐임 → 그 동은 미충족 시장이므로 출점 후보에 오름

2. 더 있는데 아무도 등록하지 않았음 → 후보로 올리면 이미 포화된 상권에 들어가는 결정이 됨

☐ 지도만 보서는 두 경우를 구분할 수 없음 — 어느 쪽이든 화면에는 점 두 개가 찍혀 있을 뿐임

○ 공공에서도 같은 오류가 반대 방향으로 발생함 — 자료의 공백을 서비스의 공백으로 읽으면 이미 시설이 있는 지역에 예산이 배정됨

☐ 구분하려면 같은 대상을 다른 방식으로 센 자료가 하나 더 필요하고, 두 자료를 대조해 얻는 값이 이 장 후반의 **포착률**(coverage rate)임

○ 이 장은 이 문제를 실제 서울 데이터로 끝까지 풀 — 자료를 찾아 받고(4~5절), 그 자료가 현실을 얼마나 담았는지 재고(6~7절), 그 결과로 결정을 내려도 되는지 판정함(8절)

3. 데이터 확보의 일곱 단계

○ 자료 확보는 순서가 있는 절차임. 앞 단계를 건너뛰면 뒤 단계의 판단 근거가 사라짐

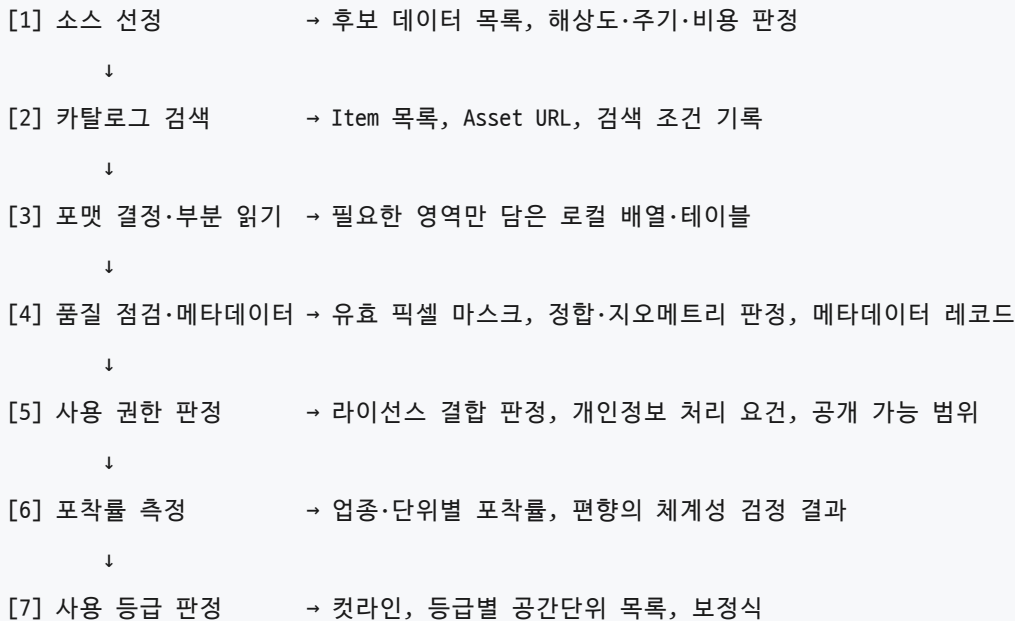


그림 3.1 데이터 확보의 일곱 단계와 단계별 산출물

○ 이 장은 일곱 단계 가운데 2단계(4절), 3단계(5절), 6단계(6~7절), 7단계(8절)를 실습으로 끝까지 실행함

□ 6·7단계는 모든 자료에 필요한 것이 아님. 자료가 만들어지는 방식에 따라 두 부류로 갈림

○ **센서가 훑는 자료**(위성영상) — 촬영된 픽셀 하나하나가 관측 대상 전체임. 구름에 가려 값이 없는 픽셀은 "관측 조건이 나빴다"는 뜻이지 "그 자리에 지표가 없다"는 뜻이 아님. 누락된 대상을 찾아 비교할 별도 명단이 없으므로 포착률을 계산하지 않음

○ **사람이 등록하거나 결제 수단에 잡히는 자료**(지도 POI, 카드 매출, 유동인구) — 등록된 것, 결제된 것만 담김. 현실의 일부만 담으므로 6·7단계가 필요함

○ 새 자료를 받으면 이 구분부터 함. 이 구분을 건너뛰고 POI 자료를 위성영상처럼 다루면, 2절의 카페 두 곳을 "정말 두 곳"으로 읽는 오류가 그대로 발생함

□ 4절부터 8절까지는 설명과 실습을 나누지 않음 — 각 절 첫머리에 실행할 명령이 나오고 그 아래에 코드와 실제 실행 결과가 이어짐. 순서대로 따라가면 이 장의 모든 수치를 직접 만들게 됨

□ 먼저 패키지를 설치함. GPU는 필요 없고 노트북에서 끝까지 돌아감

```
pip install -r lecture_practice/requirements-student.txt
```

□ 이 장에서 실행할 코드는 다섯 개이며 순서가 정해져 있음

순서	실행할 파일	하는 일	절
1	3-1-stac-search.py	STAC 카탈로그 검색	4.2
2	3-3-cog-partial-read.py	COG 부분 읽기 실측	5.2
3	3-2-cloud-formats.py	포맷별 크기·속도 비교	5.3
4	3-0b-commercial-bias-data-prep.py	포착률 분석용 집계 자료 준비	6
5	3-4-commercial-data-bias.py	포착률·편향 진단·등급 판정	6~8

- 5번은 4번이 만든 집계 파일을 입력으로 받으므로 **4번을 먼저 실행**해야 함
- 1·2·4번은 원격 카탈로그와 API를 조회하므로 인터넷이 필요하고, 3·5번은 인터넷 없이 돌아감
- 본문에 인용한 출력은 모두 `lecture_practice/chapter3/results/` 의 실행 로그에서 그대로 가져온 것이며, 어느 파일에서 왔는지를 출력 블록 첫 줄에 적어 두었음 — 화면에 나온 값과 그 파일을 한 줄씩 대조하면 됨

4. 카탈로그 검색: STAC

4.1 왜 공통 규격이 필요한가

- 위성영상 메타데이터는 센서마다 형식이 달랐음 — Landsat은 MTL 텍스트, Sentinel-2는 XML, MODIS는 HDF 내부 속성
- 같은 조건("구름 10% 미만, 이 사각형 안, 이 기간")을 센서마다 다르게 표현하고 별도 파싱 코드를 짜야 했음
- **STAC**(SpatioTemporal Asset Catalog)는 이 문제를 풀기 위한 오픈 표준임. 네 객체가 위에서 아래로 포함 관계를 이룸

객체	역할	예시
Catalog	최상위 컨테이너	Planetary Computer 전체
Collection	같은 유형 Item의 그룹	<code>sentinel-2-l2a</code>
Item	개별 관측 단위(한 장면의 메타데이터)	특정 날짜·타일의 장면
Asset	Item이 가리키는 실제 파일	<code>B04.tif</code> , <code>SCL.tif</code>

- Item은 영상 파일이 아니라 **같은 장면에 속한 파일들을 묶는 색인**임 — 관측 시각, 구름 비율, 공간 범위를 기록하고 실제 파일은 Asset으로 가리킴
- Catalog의 API 주소(<https://planetarycomputer.microsoft.com/api/stac/v1>)를 브라우저에 붙여 넣으면 JSON 응답이 뜸 — Catalog가 실제로 무엇을 담는지 바로 확인할 수 있음

4.2 검색 절차와 실제 결과

□ 먼저 실행함. 이 절의 숫자는 전부 이 명령의 출력임

```
python lecture_practice/chapter3/code/3-1-stac-search.py
```

□ 코드는 여섯 단계를 순서대로 밟음

1. **엔드포인트 접속** — Catalog API를 열고 Collection 목록을 받음
 2. **Collection 선택** — 처리 수준을 정함. `sentinel-2-l1c` 는 대기 상단 반사율, `sentinel-2-l2a` 는 대기보정을 마친 지표 반사율. 지표 상태를 보려면 L2A를 고름
 3. **조건 설정** — bbox(관심 지역을 감싸는 사각형의 서·남·동·북 좌표 네 개), datetime(기간), query(품질 조건)를 지정함
 4. **Item 목록 수신** — 조건에 맞는 장면의 메타데이터를 받음
 5. **Asset URL 획득** — 필요한 밴드 파일의 주소를 꺼냄
 6. **검색 조건과 Item ID 기록** — 엔드포인트, Collection, bbox, 기간, 품질 조건, 반환 Item ID를 전부 남김
- 아래는 그 코드에 실제 실행 결과를 주석으로 붙인 것임

```

# 주석에 붙인 출력의 출처: lecture_practice/chapter3/results/3-1-stac-search.log
from pystac_client import Client

# [1] 엔드포인트 접속 - 인증 없이 검색만 수행함
catalog = Client.open("https://planetarycomputer.microsoft.com/api/stac/v1")

for collection in catalog.get_collections():
    if "sentinel" in collection.id.lower():
        print(collection.id, collection.title)

# 출력: sentinel-1-rtc: Sentinel 1 Radiometrically Terrain Corrected (RTC)
#       sentinel-2-l2a: Sentinel-2 Level-2A
#       ... 총 16개 Sentinel 컬렉션 발견

# [2]~[3] Collection 선택과 조건 설정
search = catalog.search(
    collections=["sentinel-2-l2a"],          # 대기보정을 마친 지표 반사율
    bbox=[126.8, 37.4, 127.2, 37.7],      # 서울 일대(서·남·동·북)
    datetime="2024-06-01/2024-08-31",
    query={"eo:cloud_cover": {"lt": 10}},  # 구름 10% 미만
)

# [4] Item 목록 수신
items = list(search.item_collection())
for i, item in enumerate(items):
    print(i + 1, item.datetime, item.properties["eo:cloud_cover"])

# 출력: 검색 결과: 5개 영상
#       [1] 2024-08-29 02:15 | 구름: 8.1% | 플랫폼: Sentinel-2B
#       [2] 2024-08-29 02:15 | 구름: 0.0% | 플랫폼: Sentinel-2B
#       [3] 2024-08-19 02:15 | 구름: 9.9% | 플랫폼: Sentinel-2B
#       [4] 2024-08-14 02:16 | 구름: 7.0% | 플랫폼: Sentinel-2A
#       [5] 2024-08-04 02:15 | 구름: 9.0% | 플랫폼: Sentinel-2A
#       평균 구름 비율: 6.8% | 구름 5% 미만: 1개

# [5] Asset URL 획득 - 적색 밴드(B04) 파일 주소
item = items[0]
href = item.assets["B04"].href

# 출력: ID: S2B_MSIL2A_20240829T021539_R003_T52SCG_20240829T043313
#       좌표계: EPSG:32652 | 구름비율: 8.1%
#       Asset 목록 (23개): A0T B01 B02 B03 B04 ... SCL WVP visual

```

○ `results/3-1-stac-search.log` 는 코드 마지막 줄의 문구를 고치기 전에 저장한 실행 기록임 — 개수·구름 비율 등 숫자는 지금 코드와 같고 마지막 한 줄의 표현만 다름

□ 화면에 나온 값에서 읽을 것

○ 여름 석 달을 통틀어도 구름 10% 미만이 5개뿐이고 그중 5% 미만은 1개임 — 한국 여름은 장마·태풍 때문에 맑은 장면이 드물

○ 같은 날짜(2024-08-29)의 Item이 두 개 나옴. 서울이 두 Sentinel-2 타일 경계에 걸쳐 있어 타일마다 별도 Item이 생기기 때문이며, 관심 영역이 타일 경계에 걸리면 한 시점에도 Item 두 개를 이어 붙여야 함

○ Item 하나가 Asset을 23개 담음 — 밴드별 `.tif` 와 메타데이터 XML이 함께 들어 있고, 적색 밴드가 필요하면 `item.assets["B04"].href` 에서 그 파일 주소만 꺼냄

○ 6단계는 코드가 대신 해 주지 않음. 엔드포인트·Collection·bbox·기간·구름 조건과 반환된 Item ID를 직접 적어 남김

□ 코드는 마지막에 같은 조건으로 Landsat C2 L2도 검색함 — 결과는 6개(구름 3.1 / 6.9 / 8.2%)로 Sentinel-2의 5개보다 많음

○ 재방문 주기만 보면 Sentinel-2가 5일, Landsat이 16일이므로 Sentinel-2가 더 많이 잡혀야 하지만, 검색 개수를 정하는 것은 관측 횟수가 아니라 그 관측 가운데 조건을 통과한 장면의 수임

○ 구름 10% 미만 조건이 여름 장면 대부분을 걸러 내므로 주기가 짧다는 사실이 그대로 개수로 이어지지 않고, `max_items=20` 도 반환 개수에 상한을 두지만 이번 실행은 5개·6개여서 상한에 닿지 않았음 — **사양에 적합한 주기가 아니라 실행이 반환한 개수를 해석의 근거로 삼음**

- 라이브 카탈로그 조회는 재현되지 않음
- 아카이브가 갱신되거나 장면이 재처리되면 같은 조건에서도 반환 개수가 달라짐
- 실행했을 때 5개가 아니어도 정상이며, 그래서 6단계의 기록이 필수임 — 보고서에는 "5개가 검색되었다"가 아니라 실제로 쓴 Item ID 목록을 실음

5. 부분 읽기: 필요한 조각만 받는다

5.1 원리

○ 4절에서 얻은 Asset URL은 클라우드에 놓인 파일을 가리킴 — Sentinel-2 한 밴드가 수백 MB인데, 실제로 필요한 것은 관심 영역의 일부 픽셀뿐임

□ 전통 포맷(GeoTIFF·Shapefile 등)은 파일을 처음부터 순서대로 읽는 방식을 가정함 — 10GB 영상에서 100×100 픽셀만 필요해도 전체를 받아야 했음

○ **HTTP Range Request**가 이 전제를 깬다 — "이 파일의 몇 번째 바이트부터 몇 번째까지만 보내 달라"고 요청하는 HTTP 기능

□ 부분 읽기는 세 단계로 진행함

1. 파일 앞부분의 **헤더**만 받아 내부 구조(전체 해상도, 타일 크기, 각 타일의 바이트 위치)를 읽음 — 이 단계에서 픽셀 본문은 전송되지 않음 ○ 헤더를 개념적으로 표현하면 다음과 같음. 실제 COG 헤더의 바이너리 구조를 단순화한 가상 예시임

```
{
  "width": 10000,
  "height": 8000,
  "tile_width": 256,
  "tile_height": 256,
  "tiles": {
    "row=12,col=34": {"offset": 1843200, "length": 52781},
    "row=12,col=35": {"offset": 1895981, "length": 49120}
  }
}
```

- 이 예시에서 영상 전체 크기는 10,000×8,000 픽셀이고, 한 타일은 256×256 픽셀임. row=12,col=34 타일은 파일의 1,843,200번째 바이트부터 52,781바이트를 차지하므로, 해당 구간만 요청할 수 있음
2. 관심 영역의 지리좌표를 내부 타일 번호로 바꿈
 3. 해당 타일의 바이트 구간만 요청해 압축을 풀어 배열로 조립함

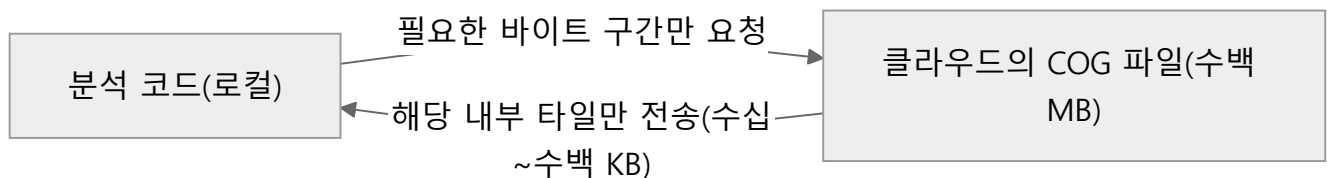


그림 3.2 부분 전송의 흐름 — 전체 파일을 내려받지 않고 관심 영역이 걸치는 내부 타일의 바이트 구간만 주고받음. COG-Zarr-GeoParquet가 공통으로 이 구조를 씀

○ 이 구조가 성립하는 조건은 둘 — 서버가 Range 요청을 지원하고, 파일 내부가 독립적으로 압축 해제 가능한 구간으로 나뉘어 있어야 함. 하나라도 없으면 파일 전체를 받아야 함

5.2 COG로 실측하기

○ **COG**(Cloud Optimized GeoTIFF)는 GeoTIFF의 내부 배치를 부분 읽기에 맞게 정리한 래스터 포맷 — 픽셀을 256×256 또는 512×512의 **내부 타일** 단위로 저장하고, 각 타일의 바이트 위치를 헤더에 적어 둠

□ 두 번째 코드를 실행함. 4절과 같은 조건으로 장면을 고른 뒤 강남 소구역만 잘라 받음

```
python lecture_practice/chapter3/code/3-3-cog-partial-read.py
```

```

# 주석에 붙인 출력의 출처: lecture_practice/chapter3/results/3-3-cog-partial-read.log
import time
import rasterio
from rasterio.windows import from_bounds
from urllib.request import Request, urlopen

# [1] 전체 파일 크기 - HTTP HEAD로 헤더만 물어봄(본문은 받지 않음)
with urlopen(Request(href, method="HEAD")) as resp:
    total_bytes = int(resp.headers["Content-Length"])
# 출력: 전체 파일 크기: 268.5 MB

# [2] 전체 해상도 - rasterio.open()은 COG 헤더만 읽음(픽셀은 아직 없음)
with rasterio.open(href) as src:
    full_width, full_height = src.width, src.height
# 출력: 전체 크기: 10980행 × 10980열 (120,560,400픽셀) | 좌표계: EPSG:32652

# [3] 관심 영역만 윈도우 읽기 - 여기서 처음으로 픽셀이 전송됨
t0 = time.time()
with rasterio.open(href) as src:
    window = from_bounds(x0, y0, x1, y1, src.transform)
    win_data = src.read(1, window=window)
# 출력: 읽은 영역: 216행 × 270열 (58,320픽셀)
#      소요 시간: 1.44초
#      픽셀 최소~최대값: 1012~8976
#      실제로 받은 픽셀: 58,320개 (전체의 0.048%)
#      저장: chapter3/data/gangnam_b04_window.tif (91.0 KB)

```

□ 화면에 나온 값에서 읽을 것

- 파일 전체는 268.5MB 에 10,980행×10,980열(120,560,400픽셀)인데, 이 값은 헤더 조회만으로 얻었으므로 이 단계에서 픽셀 본문은 오지 않음
- 실제로 받은 것은 216행×270열 = 58,320 픽셀로 전체의 **0.048%**이고 1.44초 에 도착함
- 소구역만 저장하면 91.0KB — 268.5MB 파일은 이 과정에서 한 번도 로컬에 통째로 저장되지 않음
- 픽셀값 범위 1,012~8,976은 자료가 정상적으로 왔다는 확인임. Sentinel-2 L2A가 반사도를 10,000배 정수로 저장하기 때문이며, 실수로 바꾸면 0.101~0.898에 해당함
- 0.048%라는 픽셀 비율이 전송 바이트 비율과 같지는 않음 — 서버는 타일 단위로 응답하므로, 관심 영역이 타일 경계에 조금이라도 걸치면 그 타일 전체가 전송됨. 그래도 전체 대비 전송량이 크게 줄어드는 구조는 같음
- 코드는 이어서 Green(B03)-Blue(B02)를 같은 창으로 2.45초 에 더 받아 RGB로 합성하고, 2~98% 구간으로 명암을 늘려 PNG 두 장을 남김. 받은 조각이 실제로 무엇인지 여기서 눈으로 확인함



그림 3.3 268.5MB 파일에서 0.048%만 잘라 받은 강남 소구역(216행×270열). 가시광선 세 밴드를 RGB로 합성하고 명암을 늘려 표시한 것이며, 격자 모양 도로망과 블록 단위 건물이 구분됨. **전체 장면을 내려받지 않고도 이 판독이 가능하다는 것이 부분 읽기의 요점임**

5.3 포맷은 자료의 차원 구조가 정한다

포맷	자료 유형	확장자	부분 읽기 단위	적합한 자료
COG	래스터(2차원)	.tif	내부 타일	단일 시점 위성영상, DEM
Zarr	다차원 배열	.zarr	청크	시계열, 기후모델 출력
GeoParquet	벡터 테이블	.parquet	컬럼·행 그룹	대용량 POI, 건물, 점포

○ 판정 순서: ① 벡터인가 래스터인가 → 벡터면 GeoParquet ② 래스터면 축이 2개인가 3개 이상인가 → 시간 축이 붙으면 Zarr, 단일 시점 2차원이면 COG

□ 세 번째 코드를 실행함. 같은 5만 개 포인트를 세 포맷으로 저장했다가 다시 읽어 크기와 시간을 잴

```
python lecture_practice/chapter3/code/3-2-cloud-formats.py
```

```

# 주석에 붙인 출력의 출처: lecture_practice/chapter3/results/3-2-cloud-formats.log
import geopandas as gpd

# [1] 같은 자료를 세 포맷으로 저장
gdf.to_file(path_json, driver="GeoJSON")
gdf.to_file(path_shp)
gdf.to_parquet(path_pq)

# 출력: GeoJSON      11.89 MB | 저장 1.631초
#      Shapefile    21.05 MB | 저장 1.661초
#      GeoParquet    2.18 MB | 저장 0.076초

# [2] 다시 읽어 시간 비교
gdf = gpd.read_parquet(path_pq)

# 출력: GeoJSON 0.723초 | Shapefile 0.540초 | GeoParquet 0.055초
#      GeoParquet가 GeoJSON보다 13.2배, Shapefile보다 9.8배 빠름

# [3] 행 그룹마다 덮는 공간 범위를 적어 두고, 그 범위로 걸러 읽기
gdf.to_parquet(path_pq_bbox, write_covering_bbox=True)
gdf_filtered = gpd.read_parquet(path_pq_bbox, bbox=gangnam_bbox)

# 출력: 전체 50,000개 중 2,055개 (강남 영역, 4.1%)
#      필터링 시간: 0.030초

```

□ **GeoParquet**는 값을 행이 아니라 컬럼 단위로 모아 저장하는 벡터 포맷이며, 위 출력이 그 차이를 숫자로 보여 줌

○ 파일 크기: GeoJSON 11.89MB, Shapefile 21.05MB, **GeoParquet 2.18MB**

○ 읽기 시간: GeoParquet 0.055초로 GeoJSON(0.723초)보다 13.2배, Shapefile(0.540초)보다 9.8배 빠름 — 좌표를 텍스트가 아니라 이진수로 저장하고 컬럼 단위로 압축하기 때문

○ 행 그룹마다 덮는 공간 범위를 적어 두는 bbox 정보를 쓰면 강남 영역 2,055개(4.1%)만 0.030초에 걸러 냄

○ 시간 값은 디스크와 CPU에 따라 달라짐 — 절대 시간이 아니라 포맷 사이의 배수 관계가 유지되는지를 봄. 파일 크기는 환경과 거의 무관하게 같게 나옴

- 확장자만으로 COG 여부를 판정할 수 없음

- COG와 일반 GeoTIFF는 둘 다 .tif 를 쓰고 내부 구조만 다름 — 일반 GeoTIFF에 부분 읽기를 시도하면 파일 전체가 전송됨

- rio cogeo validate 나 gdalinfo 로 내부 구조를 먼저 확인함

6. 포착률: "없다"를 재는 값

- 여기서부터 2절의 문제로 돌아감. 사람이 등록해야 잡히는 자료가 현실을 얼마나 담았는지 재는 절차임
- 사용 자료 — 서울의 상업 공간 데이터 두 갈래를 같은 행정동 위에 올림
- **상가(상권)정보**(소상공인시장진흥공단, 2026년 6월): 서울 554,092개 점포. 사업자 등록에 연동되어 분기마다 공개됨
- **OSM POI**(Overpass API 조회): 자원봉사자가 등록한 시설 위치
- 경계: 서울 행정동 425개(EPSG:5181)
- 네 번째 코드를 실행함. 세 출처를 행정동 단위 집계표로 바꾸는 준비 단계이며, 뒤의 모든 계산이 이 결과를 입력으로 씀

```
python lecture_practice/chapter3/code/3-0b-commercial-bias-data-prep.py
```

```
# 출처: lecture_practice/chapter3/results/3-0b-commercial-bias-data-prep.log
[1] 서울 행정동 경계 - 행정동 425개, 좌표계 EPSG:5181
    면적 중앙값 0.97 km², 시청 거리 0.4~17.6 km
[2] 상가(상권)정보 - 서울 점포 554,092개
    좌표로 배경: 경계 안 554,092개, 행정동 425개
    (자료에 적힌 행정동 코드와 다르게 배경된 점포 5,672개, 1.02%)
[3] OpenStreetMap POI - 40,528개 중 대응표에 잡힌 40,211개
    서울 행정동 안에 떨어진 것 38,659개
[5] 저장 - bias_dong.parquet / bias_counts.parquet / bias_sales.parquet
```

- 출력의 1.02%를 그냥 넘기지 않음 — 상가정보에 적힌 행정동 코드와 좌표로 다시 찍은 행정동이 점포 5,672개에서 어긋남. 코드가 아니라 좌표를 기준으로 삼았으므로 뒤의 집계는 모두 좌표 기준임
- 이 코드는 원자료를 받아 집계표 세 개만 남기고 원자료 자체는 저장소에 넣지 않음 — 3절 일곱 단계의 5단계(사용 권한 판정)를 이 장 스스로에게도 적용한 것임

6.1 기준 자료를 정한다

- 포착률은 나뉠셈이므로 분모가 될 자료가 필요함. 이 자료를 **준모집단 기준**(reference frame)이라 함 — 참값은 아니지만 빠짐이 가장 적어 비교의 기준선으로 삼는 자료
- 이 사례에서는 상가정보를 기준으로 삼음 — 등재가 자발적 기여가 아니라 사업자 등록이라는 제도적 절차를 따르고, 점포마다 좌표·업종이 붙어 있으며, 분기마다 갱신됨
- 기준 자료도 모집단은 아님 — 폐업 신고가 늦으면 이미 닫은 점포가 남고, 무허가 영업은 처음부터 없음

포착률 = OSM에서 확인된 카페 수
÷ 상가정보에 등록된 카페 수

○ 그래서 계산 결과는 "OSM이 실제의 22%를 담는다"가 아니라 ***"OSM이 상가정보 대비 22%를 담는다"***로 읽어야 함. 이 한계가 실제로 드러나는 장면이 다음 절에 나옴

6.2 업종을 대응시키고 계산한다

□ 두 자료의 분류 체계는 일대일로 맞지 않음 — OSM은 `amenity=shop` 같은 자체 태그, 상가정보는 대·중·소 3단계 분류

○ 어느 태그를 어느 분류에 대응시키느냐가 분자와 분모를 동시에 바꿈. 그래서 대응표를 두 판본으로 만들

– 좁은 대응: 이름이 정확히 겹치는 항목만 (카페 ↔ `amenity=cafe`)

– 넓은 대응: 성격이 가까운 항목을 더 넣음 (카페 + 아이스크림/빙수 ↔ `amenity=cafe` + `shop=coffee`)

– 두 판본을 나란히 계산하면 대응 규칙이 결과를 얼마나 흔드는지가 숫자로 나옴

□ 다섯 번째 코드를 실행함. 6절부터 8절까지의 모든 표가 이 한 번의 실행에서 나옴

```
python lecture_practice/chapter3/code/3-4-commercial-data-bias.py
```

○ 출력은 세 덩어리로 나뉨 — [A] 포착률(6~7절), [B] 카드 매출의 결측 구조, [C] 컷라인과 등급 판정(8절)

○ 6절부터 8절까지의 표와 출력 블록은 모두 이 실행의 로그 `lecture_practice/chapter3/results/3-4-commercial-data-bias.log` 에서 옮긴 것이며, 아래 표는 그중 [A-1] 구간임

○ 포착률 = OSM이 담은 점포 수 ÷ 상가정보의 점포 수. 1에 가까울수록 빠짐이 적고, 0.1이면 열에 아홉을 놓쳤다는 뜻

업종군	상가정보(좁음)	OSM(좁음)	포착률(좁음)	포착률(넓음)
카페	22,739	4,237	0.186	0.178
제과점	6,119	616	0.101	0.094
편의점	9,395	6,696	0.713	0.713
미용·뷰티	20,278	25,324	1.249	0.757
약국	5,611	698	0.124	0.129
안경	2,136	97	0.045	0.045
화장품	5,517	192	0.035	0.039
서점	1,524	113	0.074	0.074
세탁소	3,869	270	0.070	0.060
8개 합계(미용·뷰티 제외)	56,910	12,919	0.227	0.217

□ 표에서 가장 먼저 처리할 값은 미용·뷰티의 **1.249**임 — OSM이 실제보다 25% 더 많이 담았다는 뜻이 되므로 물리적으로 성립하지 않음

○ 원인: OSM의 `shop=hairstylist` 태그가 미용실만 가리키지 않음. 네일숍·피부관리실이 같은 태그로 등록되어 분자에는 여러 업종이 섞였는데 분모에는 미용실만 들어감

○ 상가정보 쪽을 네일숍·피부관리실까지 넓히면(넓은 대응) 0.757로 내려감 — 자료가 이상한 것이 아니라 대응이 어긋난 것

○ 이 업종군은 8개 합계에서 제외함. 태그 의미가 깨진 데다 개수가 가장 많아 합계를 이 하나가 좌우하기 때문

○ 나머지 여덟 업종군은 편의점만 0.713으로 높고, 카페 0.186부터 화장품 0.035까지 낮게 흩어짐 — 편의점은 브랜드가 몇 개로 정해져 있고 간판이 표준화되어 등록하기 쉽고, 화장품·안경은 간판만으로 업종을 판정하기 어렵기 때문으로 보임

○ 서울에서조차 OSM POI는 상가정보 대비 다섯 곳 중 한 곳 남짓(0.227)만 담음

- 포착률이 1을 넘으면 자료가 아니라 대응표의 문제
- 태그 하나에 여러 업종이 담기면 분자가 부풀고, 기존 자료의 분류를 좁게 잡으면 분모가 줄어듦
- 1 초과가 나오면 계산식을 고치지 말고 대응표를 다시 잡으며, 좁은·넓은 두 판본의 차이를 결과와 함께 보고함

7. 빠짐의 방향: 무작위가 아니다

- 6절은 빠짐의 크기를 잴음. 이 절은 그 빠짐이 무작위 잡음인지, 지역을 따라 움직이는지를 가름
- 구분이 중요한 이유 — 무작위라면 여러 동을 합치는 것으로 완화되지만, 지역을 따라 움직인다면 특정 지역이 체계적으로 어둡게 나오므로 그대로 쓰면 지역 비교 자체가 왜곡됨
- 행정동마다 8개 업종군을 합쳐 포착률을 계산함. 분모가 30개 미만인 13개 동은 관측 하나에 비율이 크게 튀므로 제외하고 412개 동을 봄
- 분포: 최소 0.000 / 중앙값 0.190 / 최대 1.069
- 가장 높은 동: 가회동(1.07), 교남동(0.86), 삼청동(0.70) — 도심과 관광지
- 가장 낮은 동: 상계8동(0.00), 문정2동(0.03), 시흥2동(0.03) — 외곽 주거지
- 시청까지의 거리로 회귀하면 거리 효과만 뚜렷하게 확인됨(계수 -0.3089 , $p = 2.27 \times 10^{-13}$) — **빠짐은 무작위가 아니라 도심에서 멀수록 커짐**
- 회귀 출력은 같은 실행의 [A-3] 구간에 그대로 찍힘

출처: lecture_practice/chapter3/results/3-4-commercial-data-bias.log [A-3]

항	계수	표준오차	z	p
const	-1.3325	0.0330	-40.38	0.00e+00
log_점포밀도	-0.0605	0.0506	-1.19	2.32e-01
시청거리_km	-0.3089	0.0421	-7.33	2.27e-13
log_면적	0.0621	0.0527	1.18	2.38e-01

이탈도(deviance) 3,769 / 자유도 408 → 분산 팽창 9.2배 (과대산포)

잔차의 공간 자기상관 Moran's I = 0.262 (기댓값 -0.002 , 순열검정 $p = 0.001$)

- 비율을 그대로 최소제곱에 넣지 않고 점포 하나하나의 포착 여부를 성공·실패로 보는 이항 GLM을 씀 — 동마다 분모가 크게 다르기 때문임
- 분산 팽창 9.2배와 Moran's I 0.262가 함께 나온 것은, 세 설명변수로 걸러지지 않은 차이가 남아 있고 그 나머지가 특정 구역에 뭉쳐 있다는 뜻임. 동을 서로 독립으로 놓고 세운 추론은 그만큼 위태로움

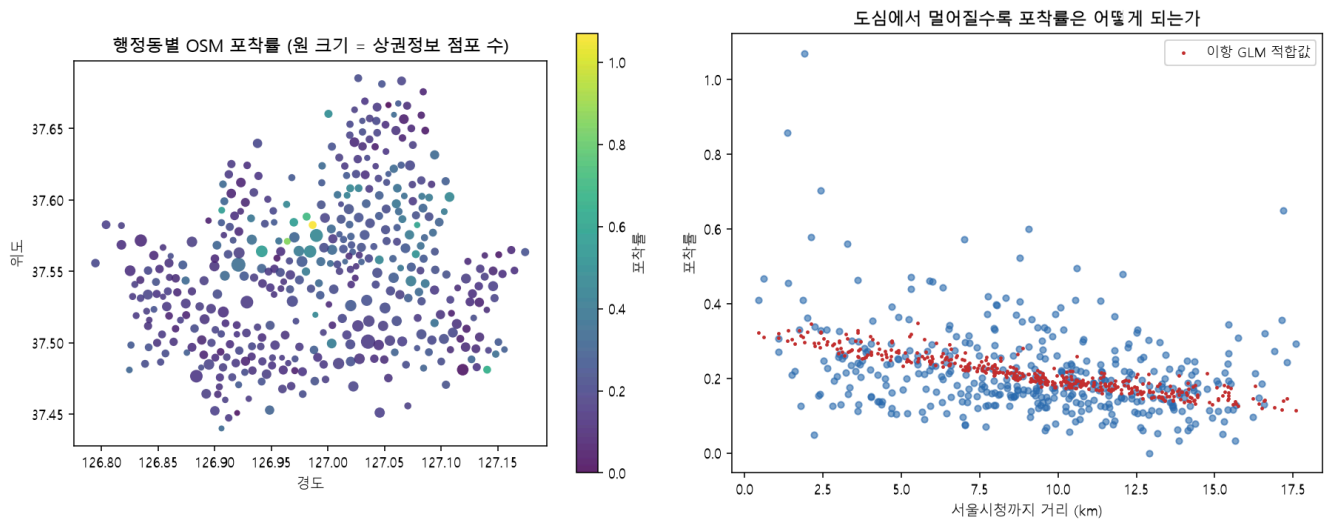


그림 3.4 왼쪽은 행정동 중심점에 찍은 OSM 포착률이며 밝을수록 높고 원 크기는 상가정보 점포 수 — 시청과 종로·중구 일대가 밝고 외곽으로 갈수록 어두워짐. 오른쪽은 서울시청까지 거리에 따른 포착률과 회귀 적합값으로, 거리가 늘수록 적합값이 0.30에서 0.12 부근까지 내려감. 같은 거리에서도 점이 위아래로 크게 흩어지는데, 거리만으로 설명되지 않는 동별 차이가 그만큼 크다는 뜻임

□ 6.2의 대응표 문제가 여기서 다시 나타남 — 합계에서 뺀 미용·뷰티를 다시 넣고 같은 회귀를 돌리면 거리 계수의 부호가 뒤집히고 유의성이 사라짐(+0.0615, $p = 0.209$)

○ OSM 점포의 3분의 2가 이 업종군이어서, 합계의 값이 사실상 이 업종군 하나로 정해지기 때문

○ **대응표를 어떻게 잡았는지 공개하지 않으면 결론을 검증할 수 없음** — 1 초과를 확인하고 처리한 기록이 결론의 근거가 됨

□ 다만 좁은·넓은 대응으로 각각 낸 동별 포착률의 순위상관은 0.994 — 대응을 넓혀도 동 사이의 순서는 거의 그대로임

○ 그래서 이 절의 결론은 "포착률이 몇 %다"가 아니라 *****어느 동이 더 낫다*****로 세움 — 절대 수준은 대응표와 기준 자료에 얹혀 있지만, 순서는 안정적인

8. 지도의 "0개"는 정말 0개인가

8.1 사후확률: 0개 관측의 의미를 계산한다

○ 2절의 질문을 이제 숫자로 풀. 포착률이 p 인 자료에서 점포 하나가 등록될 확률을 p 로 보면, 실제 점포가 N 개인 동이 지도에 0개로 보일 확률은 $(1-p)^N$ 임

□ **서점**(실측 포착률 0.074)으로 계산해 보면

○ 실제 서점이 3개인 동이 지도에 0개로 보일 확률: $(1-0.074)^3 = 0.926^3 = \mathbf{0.794}$ — 열 곳 중 여덟 곳에서 세 개가 전부 빠짐

○ 서점이 10개여도 $0.926^{10} = 0.462$ — 절반 가까이가 빈 것처럼 보임

○ 여기에 "애초에 서점이 0개인 동이 얼마나 되는가"(사전확률, 상가정보 기준 $P(N=0) = 0.118$)를 붙여 베이즈 규칙으로 뒤집으면, **0개로 보이는 동이 정말 0개일 확률(사후확률)**이 나옴

$$P(\text{진짜 0개} \mid \text{0개로 보임}) = P(N=0) \div \sum_n P(N=n) \cdot (1-p)^n$$

○ 서점의 사후확률은 **0.150** — 지도에 서점이 0개로 보이는 동 일곱 곳 가운데 여섯 곳은, 서점이 있는데 등록되지 않았을 뿐임

□ 코드는 포착률을 바꿔 가며 사후확률이 어떻게 움직이는지도 함께 찍음([C-1] 구간)

출처: lecture_practice/chapter3/results/3-4-commercial-data-bias.log [C-1]

자세히 보기 - 서점 (점포 0개인 동 50개, 중앙값 3개)

포착률 p	P(0개로 보임)	P(정말 0개 0개로 보임)	판정
0.05	0.845	0.139	판정 불가
0.10	0.725	0.162	판정 불가
0.20	0.550	0.214	판정 불가
0.30	0.429	0.274	판정 불가
0.50	0.277	0.425	판정 불가
0.70	0.190	0.619	판정 불가
0.90	0.137	0.860	진입 가능

○ 포착률을 0.05에서 0.70까지 열네 배로 올려도 사후확률은 0.139에서 0.619까지만 오름 — 8.2절에서 정하는 컷라인 0.80을 넘기려면 포착률이 0.85는 되어야 하는데 실측은 0.074임

업종군	0개인 동	실측 포착률	사후확률	필요 p*	판정
서점	50	0.074	0.150	0.85	판정 불가
안경	33	0.045	0.096	0.84	판정 불가
화장품	6	0.039	0.021	0.90	판정 불가
약국	5	0.129	0.044	0.85	판정 불가
카페	0	0.178	0.000	—	판정 불가
편의점	0	0.713	0.000	—	판정 불가
미용·뷰티	0	0.757	0.000	—	판정 불가

○ 표 읽는 법 — **실측 포착률**은 6절에서 계산한 관측값이고, ****필요 p*****는 "0개 관측을 실제 공백으로 판정하려면 포착률이 최소 얼마여야 하는가"라는 판정 기준임. 실측이 필요치보다 낮으면 그 업종군에서는 0개 관측으로 공백을 판정할 수 없음

□ 일곱 업종군 모두에서 판정 불가가 나오지만, 막히는 이유는 둘로 갈림

○ **첫째 갈래 — 포착률이 모자라는 경우.** 서점·안경·화장품·약국이 여기에 해당함. 서점은 필요 p^* 0.85에 실측 0.074로 열 배 이상 모자람

○ **둘째 갈래 — 질문이 성립하지 않는 경우.** 카페·편의점·미용·뷰티는 서울 425개 동 전부에 점포가 실재해 "0개인 동"이 하나도 없음. 사전확률 $P(N=0)$ 이 0이므로 사후확률도 0이 되고, 포착률을 아무리 올려도 컷라인에 닿지 않아 필요 p^* 열이 —로 남음. 어디에나 깔린 업종에서는 "0개"라는 관측이 애초에 나오지 않으므로, 판정 대상을 0개 여부가 아니라 개수 수준으로 바뀌어야 함

8.2 컷라인: 판단 기준은 오판 비용이 정한다

□ 사후확률이 얼마 이상이어야 결정을 내릴 수 있는가 — 이 기준선(컷라인)은 자료가 정하지 않음. **두 가지 오판이 각각 얼마의 손실을 내는지가 정함**

○ 잘못 진입하면 인테리어·권리금 같은 매물 투자를 잃고(L진입), 진입하지 않았는데 정말 비어 있었다면 기회를 놓침(L상실). 두 기대손실이 같아지는 지점이 컷라인임

$$q^* = L_{\text{진입}} \div (L_{\text{진입}} + L_{\text{상실}})$$

□ 매물 투자 1.0, 기회비용 0.25로 두면 $q^* = 1.0 \div 1.25 = \mathbf{0.80}$ — 정말 비었다는 확률이 80% 이상일 때만 진입함

□ 공공 결정에서는 비용의 방향이 반대임 — 시설을 놓아야 할 곳을 놓치는 손실(사각지대 방치)이 1.0이고, 공백이 아닌 곳을 한 번 더 현장 확인하는 비용이 0.25라면 $q^* = 0.25 \div 1.25 = \mathbf{0.20}$

○ 컷라인이 낮아지는 만큼 판정의 내용도 바뀜 — "진입한다"가 아니라 "의심되는 동을 현장 확인 대상으로 올린다"

□ **같은 지도, 같은 포착률에서도 창업 희망자는 0.80을, 구청 담당자는 0.20을 쓰는 것이 정상임** — 1장에서 본 오류 비용의 비대칭이 여기서 컷라인이라는 숫자로 나타남

- 컷라인은 자료가 아니라 오판 비용의 비가 정함
- 판정표를 보고할 때 사용한 두 손실의 비를 함께 적어야 함 — 적지 않으면 다른 목적(공공 배치)에서 그 표를 잘못 재사용하게 됨

8.3 등급 판정표: 동마다 결론을 붙인다

□ 마지막으로 412개 동 각각에 사용 등급을 붙임. 판정 측은 둘 — ① 사후확률(그 동의 0개 관측을 믿어도 되는가) ② 그 동의 포착률이 7절 회귀식의 예측에서 얼마나 벗어났는가(|z|)

○ 두 측을 대기 전에 걸러야 할 것이 하나 있음 — 포착률이 1을 넘는 동은 확률로 해석할 수 없으므로 사후확률을 계산하지 않고 따로 뺌. 1 초과를 0.999로 잘라 넣으면 대응표가 어긋나 생긴 값이 오류 없이 계산을 통과해 가장 높은 등급을 받음

등급	기준	행정동 수	포착률 범위	대표 동
그대로 사용	포착률 ≤ 1 , 사후확률 ≥ 0.80	1	0.857	교남동
보정 후 사용	포착률 ≤ 1 , 사후확률 < 0.80 , $ z \leq 2$	248	0.089~0.471	가산동, 신사동
사용 금지	포착률 ≤ 1 , 사후확률 < 0.80 , $ z > 2$	162	0.000~0.704	역삼1동, 서교동
대응표 점검	포착률 > 1	1	1.069	가회동

○ **그대로 사용**은 412개 중 교남동 1곳뿐임(포착률 0.857) — 지도만으로 공백을 판정해도 되는 동은 서울에서도 예외에 가까움

○ **대응표 점검**은 포착률이 1을 넘어 확률로 해석할 수 없는 동임 — 가회동은 분모 101개 중 카페가 58개인데 OSM 카페는 75개임. 관광지에서 상가정보의 다른 소분류로 등록된 점포가 OSM에서는 amenity=cafe 하나로 모이면 분자만 부족. 등급을 붙이지 않고 해당 업종군의 대응 규칙을 다시 잡은 뒤 재집계함

○ **보정 후 사용**은 포착률이 낮지만 회귀식이 예측하는 만큼만 낮은 동 — 관측 개수를 예측 포착률로 나눠 실제 개수를 되돌린 뒤 씬. 예측 포착률 0.20인 동에 3개가 보이면 $3 \div 0.20 = 15$ 개 안팎으로 놓고 판단함

○ **사용 금지**는 포착률이 낮으면서 예측에서도 크게 벗어난 동 — 보정식을 적용할 근거가 없으므로, POI만으로 판정하지 않고 다른 자료를 결합하거나 현장을 확인함

○ 이 결과가 2절 질문의 최종 답임 — **서울 412개 동 가운데 지도 자료만으로 "비어 있다"를 판정할 수 있는 동은 1곳뿐임**. 248개는 보정을 거쳐야 하고 162개는 다른 자료가 필요하며, 1개는 대응표부터 다시 잡아야 함

9. 핵심 요약

○ **"자료에 없다"와 "현실에 없다"는 다른 말임**. 지도의 카페 두 곳은 "정말 두 곳"일 수도, "더 있는데 등록이 안 된 것"일 수도 있으며, 지도만으로는 구분할 수 없음 (2절)

○ **자료는 두 부류로 나뉨**. 센서가 훑는 자료(위성영상)는 그 자체가 관측 전체이고, 사람이 등록하거나 결제에 잡히는 자료(POI·카드 매출)는 현실의 일부만 담음. 포착률 측정과 등급 판정은 뒤쪽에만 적용함 (3절)

○ **STAC는 위성영상을 찾는 공통 규격**. Catalog → Collection → Item → Asset의 포함 관계이며, Item은 파일이 아니라 색인임. 라이브 조회는 재현되지 않으므로 검색 조건과 Item ID를 기록함 (4절)

○ **부분 읽기는 필요한 조각만 받는 구조**. 268.5MB 파일에서 관심 영역 0.048%만 1.44초에 받았고, 전체 파일은 한 번도 저장되지 않았음. 포맷은 자료의 차원 구조가 정함 — 2차원 래스터는 COG, 시계열은 Zarr, 벡터 테이블은 GeoParquet (5절)

○ **포착률은 기준 자료 대비 비율임**. 서울 OSM POI는 상가정보 대비 0.227, 즉 다섯 곳 중 한 곳 남짓만 담음. 포착률이 1을 넘으면(미용·뷰티 1.249) 자료가 아니라 대응표의 문제이며, 계산식이 아니라 대응표를 고침 (6절)

○ **빠짐은 무작위가 아님**. 도심에서 멀수록 포착률이 체계적으로 낮아지며(계수 -0.3089), 대응표를 바꾸면 이 결론 자체가 뒤집힐 수 있으므로 대응표를 결과와 함께 공개함. 절대 수준보다 동 사이의 순서(순위상관

0.994)가 안정적임 (7절)

○ **"0개로 보임"과 "정말 0개"는 확률로 연결됨.** 서점 포착률 0.074에서는 0개로 보이는 동 일곱 곳 중 여섯 곳에 실제로 서점이 있음(사후확률 0.150). 판단 기준선인 컷라인은 자료가 아니라 오판 비용의 비가 정함 — 출점 결정 0.80, 공공 배치 0.20 (8절)

○ **서울 412개 동 가운데 지도만으로 공백을 판정할 수 있는 동은 1곳뿐임.** 248개는 보정(관측 ÷ 예측 포착률) 후 사용, 162개는 다른 자료나 현장 확인이 필요함 (8.3절)

10. 과제

□ 이번 강의 과제는 **자료를 어디까지 믿을지 숫자로 판단하는 것**

○ 명령 한 줄이면 제출할 파일이 생성됨

```
python scripts/submit.py 3 --id 학번 --name 이름
```

□ 이 명령이 대신 처리하는 작업: 자료 준비(3-0b-commercial-bias-data-prep.py) 실행, 대표 실습 3-4-commercial-data-bias.py 실행, 제출용 파일 생성

○ 끝나면 submissions/ 폴더에 제출_3장_학번.md 파일이 생기고 실행 결과가 이미 붙어 있음

○ **눈여겨볼 곳은 업종별·동별 포착률, 그리고 1을 넘는 값이 있는지**

○ 맨 아래 빈칸 세 개를 채우면 됨

1. 눈에 띈 숫자 하나를 그대로 옮겨 적고, 왜 눈에 띄었는지 한 문장으로
2. 그 숫자가 왜 그렇게 나왔는가
3. **지도에 점포가 하나도 없는 동네가 있다. "여기는 비어 있으니 창업하자"고 말해도 되는가? 근거를 숫자로 대라.**

□ **3번이 이 과제의 핵심.** 답에는 포착률·사후확률·컷라인 가운데 적어도 두 개가 숫자로 들어가야 함 — 6~8절이 그 재료임

○ 과제의 핵심은 숫자 산출이 아니라 그 숫자를 근거로 판단하는 능력

○ 채운 파일 **하나만** 올리면 제출이 끝남

• 막혀도 제출함

- 실행이 도중에 실패하면 오류 메시지가 그 파일에 자동으로 담기고, 질문이 "어디서 막혔나"로 바뀜
- 어디서 왜 막혔는지 적은 제출도 온전한 제출. 실행 성공 여부로 점수를 매기지 않음

• 이 강의 대표 실습은 난수를 쓰지 않음

- 미리 준비된 자료를 그대로 집계하므로 교재에 적힌 값이 그대로 나와야 정상이고, 옆 사람 결과와도 같음

- 다만 STAC 검색(3-1)은 라이브 카탈로그 조회라 시점에 따라 개수가 달라질 수 있음 — 대표 실습(3-4)에는 해당하지 않음